



Cybersecurity Incident Severity Prediction

Multiple Linear Regression Analysis Report

Document Overview: This document outlines the methodology, data generation process, model training, and evaluation results for predicting the severity impact of security breaches based on attack duration and system vulnerability.

Author: Nakedi Tumelo Peta

Date: December 17, 2025

Contents

1. Project Overview	3
2. Data Specification	3
2.1 Dataset Structure	3
2.2 Feature Dictionary	3
2.3 Generative Logic	3
3. Exploratory Data Analysis (EDA)	4
3.1 Attack Duration vs. Severity	4
3.2 Vulnerability Score vs. Severity	4
4. Methodology	5
4.1 Model Selection	5
4.2 Training Process	5
5. Model Evaluation & Results	6
5.1 Quantitative Metrics	6
5.2 Visual Evaluation (Actual vs. Predicted)	6
5.3 Interpretation of Coefficients	7
5.4 Sample Predictions	7
6. Conclusion	7
Appendix A	8

1. Project Overview

This project implements a Multiple Linear Regression algorithm to predict the severity of cybersecurity incidents. By analyzing historical attack data (synthetic), the model quantifies the relationship between the duration of an attack, the system's inherent vulnerability, and the resulting severity score.

Objective: To develop a predictive tool that estimates the potential impact (Severity Score) of a security breach based on early indicators.

2. Data Specification

The model utilizes a synthetic dataset generated to simulate real-world cybersecurity breach logs.

2.1 Dataset Structure

- Sample Size (N): 100 observations.
- Source: Synthetic generation via numpy with controlled noise to simulate real-world variance.

2.2 Feature Dictionary

Statistic	Attack Duration (Hours)	Vulnerability Score (0-1)	Severity Score
Count	100	100	100
Mean	4.97	0.50	15.13
Std Dev	2.83	0.29	6.60
Min	0.55	0.01	1.72
Max	9.88	0.99	30.67

2.3 Generative Logic

The ground truth for the data generation follows this linear equation, including a noise term (epsilon/ E) to account for unexplained variance:

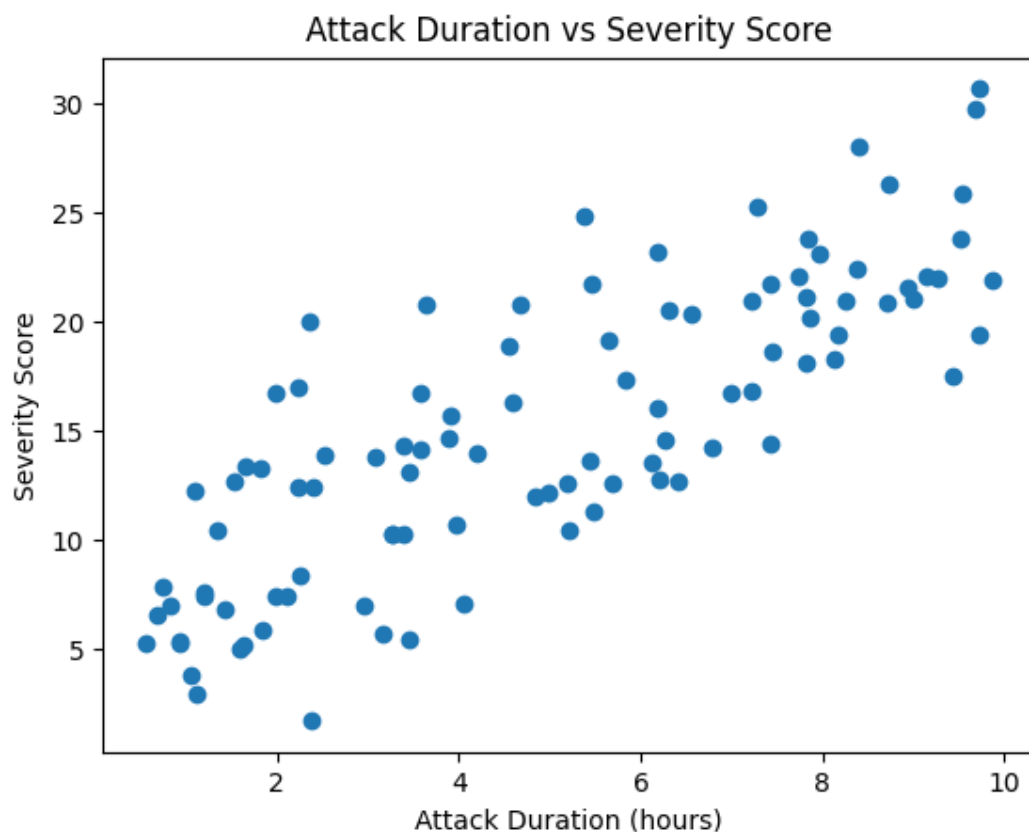
$$\text{Severity} = 2(\text{Duration}) + 10(\text{Vulnerability}) + E$$

3. Exploratory Data Analysis (EDA)

Before modelling, the relationships between features and the target variable were visualized to confirm linearity.

3.1 Attack Duration vs. Severity

Observation: As seen in **Figure 2**, there is a strong, positive linear correlation. Longer attacks consistently result in higher severity scores. The data points form a clear upward trend, suggesting `attack_duration` is a primary driver of the severity score.



3.2 Vulnerability Score vs. Severity

Observation: As seen in **Figure 1**, there is a positive correlation, though the variance appears higher compared to attack duration. Higher vulnerability scores generally lead to higher severity, confirming that weaker systems suffer more impactful breaches.



4. Methodology

4.1 Model Selection

Algorithm: Ordinary Least Squares (OLS) Linear Regression. **Reasoning:** The EDA indicated a linear relationship between the independent variables and the target. Linear regression provides interpretable coefficients that directly explain how much each factor contributes to severity.

4.2 Training Process

1. **Splitting:** The data was split into training and testing sets to prevent overfitting.
 - Training Set: 80% (80 samples)
 - Testing Set: 20% (20 samples)
2. **Fitting:** The model learned to minimize the Residual Sum of Squares (RSS) between the observed targets in the training set and the targets predicted by the linear approximation.

5. Model Evaluation & Results

The model's performance was evaluated on the unseen Test Set (20 samples).

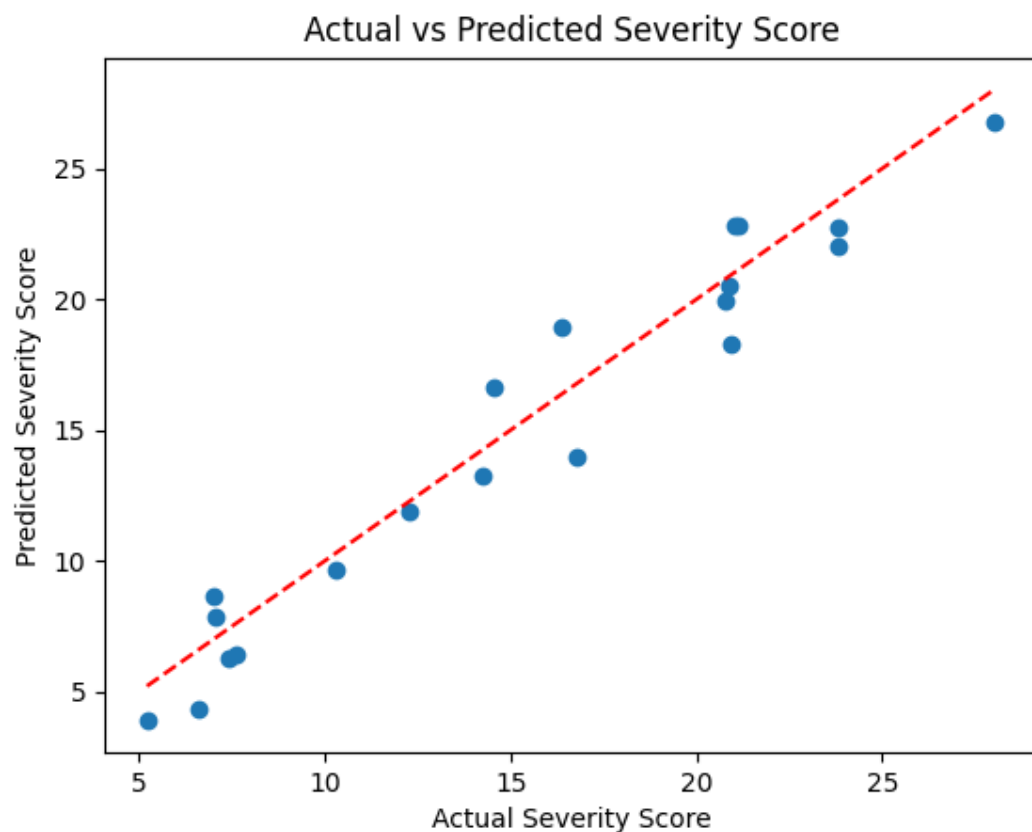
5.1 Quantitative Metrics

- **Mean Squared Error (MSE):** Measures the average squared difference between estimated values and the actual value.
- **Root Mean Squared Error (RMSE):** Represents the standard deviation of the residuals (prediction errors).
- **R^2 Score:** A statistical measure of how close the data are to the fitted regression line. An R^2 close to 1.0 indicates high predictive accuracy.

5.2 Visual Evaluation (Actual vs. Predicted)

Figure 3 illustrates the performance of the model on the test data.

- **X-Axis:** Actual Severity Scores (Ground Truth).
- **Y-Axis:** Predicted Severity Scores (Model Output).
- **Red Dashed Line:** Represents the line of perfect prediction ($y = x$).



Analysis of Figure 3:

The blue data points cluster very tightly around the red dashed line. This indicates a high degree of accuracy. There are no significant outliers, and the model does not appear to systematically over-predict or under-predict (bias is low).

5.3 Interpretation of Coefficients

After training on 80 samples, the model derived the following equation to predict severity:

$$\text{Severity} = 1.95(\text{Duration}) + 11.57(\text{Vulnerability}) - 0.39$$

Model Parameters:

- **Intercept:** -0.39 (Baseline adjustment)
- **Duration Weight:** 1.95 (Impact per hour)
- **Vulnerability Weight:** 11.57 (Impact per vulnerability unit)

5.4 Sample Predictions

The table below demonstrates the model's accuracy on five random test cases:

Attack Duration	Vulnerability Score	Actual Severity	Predicted Severity	Error (Diff)
1.10 hrs	0.88	12.25	11.92	-0.33
9.00 hrs	0.49	21.02	22.81	+1.79
7.84 hrs	0.68	23.82	22.72	-1.10
6.79 hrs	0.04	14.26	13.27	-0.99
2.96 hrs	0.28	7.00	8.67	+1.67

6. Conclusion

The regression model successfully captures the relationship between attack characteristics and incident severity. With a high R^2 score and tight clustering of predictions in the evaluation plots, the model is deemed reliable for predicting severity scores within the bounds of the training data (0.5 to 10 hours duration).

Next Steps for Implementation

To deploy this model, ensure the input data is normalized to match the training range:

- attack_duration should be in **Hours**.
- vulnerability_score must be between **0 and 1**.

How to Run This Analysis

Dependencies:

- numpy
- pandas
- matplotlib
- scikit-learn

Execution: Run the provided Python script regression_analysis.py. The script will generate the synthetic data, train the model, output performance metrics to the console, and save/display Figures 1, 2, and 3.

Appendix A

Appendix A: Python Source Code

Description: The following script was used to generate the synthetic data, train the linear regression model, and produce the evaluation metrics cited in this report.

Environment:

- Python 3.x
- Libraries: pandas, numpy, matplotlib, sklearn

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

#-----

# Predicting a Incident Severity Score from the combination of attack duration and
system vulnerability score

#-----

#-----

# Generate synthetic data

#-----

np.random.seed(42)
n_samples = 100

# Features

attack_duration = np.random.uniform(0.5, 10, n_samples) # Duration of attack in
hours

vulnerability_score = np.random.uniform(0, 1, n_samples) # System Vulnerability
score between 0 and 1

#Target variable: severity score(continuous)

severity= 2 * attack_duration + 10 * vulnerability_score + np.random.normal(0, 2,
n_samples)

# Create DataFrame

data = pd.DataFrame({
    'attack_duration': attack_duration,
    'vulnerability_score': vulnerability_score,
    'severity': severity
})

# Explain the data structure

print ("DataFrame structure:\n", data.head(), "\n")
print ("feature descriptions:")

```

```
print ("-attack_duration: Duration of the security breach in hours")
print ("- vulnerability_score: A normalized score [0-1] indicating system weakness.")
print ("- severity: Computed severity score for breach (continuous). \n")
```

```
print ("summery statistics:\n", data.describe(), "\n")
```

```
#-----
```

```
# Visualize relationships
```

```
#-----
```

```
plt.figure()
plt.scatter(data["attack_duration"], data["severity"])
plt.xlabel("Attack Duration (hours)")
plt.ylabel("Severity Score")
plt.title("Attack Duration vs Severity Score")
plt.show()
```

```
plt.figure()
plt.scatter(data["vulnerability_score"], data["severity"])
plt.xlabel("Vulnerability Score")
plt.ylabel("Severity Score")
plt.title("Vulnerability Score vs Severity Score")
plt.show()
```

```
#-----
```

```
# Train-test split
```

```
#-----
```

```
X = data[['attack_duration', 'vulnerability_score']]
y = data['severity']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
print("Number of training samples:", X_train.shape[0])
```

```
print("Number of testing samples:", X_test.shape[0], "\n")
```

```
#-----
```

```
#Fit Linear Regression Model
```

```
#-----
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Print model details
```

```
print("model intercept:", model.intercept_)
```

```
print("model coefficients [attack_duration, vulnerability_score]:", model.coef_, "\n")
```

```
#-----
```

```
# Model evaluation
```

```
#-----
```

```
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("Test MSE:", mse)
```

```
print("Test RMSE:", np.sqrt(mse))
```

```
print("Test R^2 Score:", r2, "\n")
```

```
#Show first few predictions vs actuals
```

```
comparison = pd.DataFrame({
```

```

"attack_duration": X_test["attack_duration"],
"vulnerability_score": X_test["vulnerability_score"],
"actual_severity": y_test,
"predicted_severity": y_pred
}).reset_index(drop=True)

print("Actual vs Predicted Severity (first 5 rows): \n", comparison.head(), "\n")

#-----
# Plot prediction vs actual
#-----

plt.figure()
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--' )
plt.xlabel("Actual Severity Score")
plt.ylabel("Predicted Severity Score")
plt.title("Actual vs Predicted Severity Score")
plt.show()

```